

Topological Feature Learning for Parameter Estimation in Neyman-Scott Processes

Flavio Gualtieri

August 19, 2026

Abstract

The parameters of spatial point processes are interpretable scientific quantities, but the methods available to estimate them rely on closed-form expressions for hand-picked summary statistics such as the K and g functions. This reliance limits them: a closed form must be re-derived for each new family of processes, and the choice of statistic discards information about the point pattern before any estimation takes place. We propose a deep learning estimator that instead uses vectorized persistence summaries as features, which require no closed form and no choice of summary statistic. We pursue two objectives. Firstly, we compare several topological summaries against the estimation task directly, so that their relative efficacies indicate which structure each one retains. Secondly, we use the strongest summary to build an estimator for the Thomas and nested Thomas processes under a common reparameterization, and evaluate it against classical baselines on shared test data. Our contributions include a per-diagram calibration scheme for vectorizing across a heterogeneous population of persistence diagrams, and a simulator validated against an exact finite-window second-order identity. On the Thomas process, our estimator reaches a mean loss of $\mathcal{L} = 0.126 \pm 0.008$, within noise of a specialized classical-summary neural baseline (0.115 ± 0.007) and roughly an order of magnitude below minimum contrast, which is both worse on average (0.827 ± 0.646 fit to K ; 6.167 ± 0.331 fit to g) and unstable across realizations. Because it requires no closed-form summary statistic, the same estimator applies unchanged to the harder nested Thomas process ($\mathcal{L} = 0.196 \pm 0.006$), for which no minimum-contrast baseline can be constructed at all.

1 Introduction

The study of spatial point processes finds applications in astronomy, cell biology, epidemiology, etc REF. Many natural phenomena - such as galaxy clustering REF - are modelled using SPPs, so one can hope to leverage an understanding of these processes to obtain novel insights.

The parameters of spatial point processes are of particular interest because they are interpretable scientific quantities. [TODO: For example ...]

A number of parameter estimation methods have been proposed, typically relying on closed-form expressions for summary statistics, such as the K and L functions (see Section 3.1). Minimum contrast estimation fits parameters by numerically minimizing a discrepancy between the empirical and theoretical curve, most commonly for K or g REF [Diggle 2003]. Palm likelihood instead builds a composite likelihood from the process's Palm intensity, and has been developed specifically for Neyman-Scott processes REF [Tanaka 2008]. Fully Bayesian approaches treat the unobserved parent locations as latent variables and sample the posterior over θ by MCMC REF [Moller 2004].

These methods have three limitations we hope to address.

Firstly, they rely on the existence of closed-form expressions of summary statistics which may not exist for every process. [TODO: For example ...]

Secondly, classical parameter estimation methods suffer from hyperparameter sensitivity, specifically the minimum contrast method depends on (r_{max}, q, p) . [TODO: ParamNet too. Justify]

Finally, classical methods become unstable as the process approaches complete spatial randomness (near-Poisson regime). As the overlap index ν grows the clusters merge, so κ and μ cease to be separately identifiable from a single realization: a small parent intensity with large clusters and a large parent intensity with small ones produce point patterns whose second-order behaviour is near-indistinguishable. Minimum contrast is fitted by numerical optimization of a discrepancy that is close to flat along this trade-off, so the estimates it returns in this regime are sensitive to the starting point and to the choice of r_{max} . We deliberately retain this regime in our design distribution (see Section 5) rather than restricting to well-separated clusters, and report error as a surface over the design grid so that the degradation is visible rather than averaged away.

These limitations motivate the use of persistence-based features which provide a multi-scale, process-agnostic descriptor of the point cloud that does not require a closed-form or a choice of summary statistic. [TODO: Not strictly true. Rephrase]

1.1 Contributions

Notes to expand into prose; subsection not final

- **Estimator.** One architecture and training recipe – persistence images of the H_0 diagram under a single DTM_5 filtration, a small CNN encoder, and concatenation with $\log N(\mathbf{x})$ – designed to cover Thomas and nested Thomas; on Thomas it is competitive with (not decisively better than) a specialized classical-summary neural baseline, but unlike minimum contrast it transfers unchanged to nested Thomas, where no closed-form summary statistic exists to fit against.
- **Calibration methodology.** Per-diagram coverage-quantile calibration for summarizing across a heterogeneous population of diagrams, with an explicit train/test leakage protocol (Section 6.3) and a sweep over $q, \alpha, \sigma_{\text{pixels}}, G$ showing which choices actually matter (Appendix A.1).
- **Validated simulator and evaluation protocol.** Multi-family simulator under a common reparameterization, validated against an exact finite-window second-order identity (Section 5); an evaluation protocol anchored between an upper, chance-level reference (label shuffling, Table 2) and a lower, irreducible-error-floor reference ($\mathcal{L}_{\text{floor}}$, Section 4 – currently unresolved, see Section 7.1).
- **What the estimator turns out not to need.** Multi-scale fusion, the H_1 channel, coordinate channels, and three alternative vectorizations are each reported as branches off the main line (Section 7.3); none of them earns a place in the model, and the single configuration where the simplification costs something – fused scales on nested Thomas – is reported with them rather than absorbed.

2 Related Work

REF [Vihrs 2022] first proposed the use of deep learning for parameter estimation. They implement an encoder/inference architecture similar to PointNet, but use the L -function as the learned feature

and therefore suffer from the same limitations as classical methods (i.e. reliance on a closed-form summary statistic). REF employ a similar architecture as REF, but actually use topological features. Their model is designed to estimate cosmological parameters rather than the parameters of the underlying point process. Furthermore, they do not systematically compare various topological summaries, nor do they address the calibration of the summaries they use.

The vectorization of persistence diagrams is an active area of research arising from the issue that persistence diagrams – not being a Hilbert space – cannot be directly used as inputs for machine learning models. To mitigate this, REF [Adams 2017] propose persistence images, which vectorise persistence diagrams by applying a Gaussian kernel to each point in the diagram and integrating over a grid. REF propose Betti curves, which vectorize persistence diagrams by counting the number of topological features that are alive at each scale.

3 Background

We provide the background in SPPs and persistent homology required for this discussion. Readers with a working understanding of both may skip to Section 4.

3.1 Spatial point processes

Definition 3.1: Intensity function A d -dimensional SPP X on a subset $W \subseteq \mathbb{R}^d$ is equipped with an intensity function $\rho(u) : \mathbb{R}^d \rightarrow \mathbb{R}^+$ such that

$$\mathbb{E}[N(W)] = \int_W \rho(u) du,$$

where $N(W) = |W \cap X|$ denotes the number of points of X in W . When $\rho(u) = \lambda \in \mathbb{R}$, X is said to be *homogeneous*.

Example For a homogeneous Poisson process with intensity λ , we have $\rho(u) = \frac{1}{\lambda}$, i.e. points are scattered uniformly on the space.

Definition 3.2: Neyman-Scott process Let P be a homogeneous Poisson *parent* process with intensity κ . For a Neyman-Scott process with Poisson-distributed offspring we have:

$$\rho_{NS}(u) = \mu \sum_{p \in P} k(u - p),$$

where k is a kernel that determines how the *offspring* points are distributed around the parents.

Note Neyman-Scott processes always have the parent and offspring intensities as parameters. Additional parameters are introduced by the choice of kernel.

Definition 3.3: Homogeneous Thomas process A homogeneous Thomas process is a Neyman-Scott process with an isotropic Gaussian kernel $\mathcal{N}(0, \sigma^2 I)$

$$\rho_{Thomas}(u) = \mu \sum_{p \in P} k(0, \sigma^s),$$

This process has parameters $\theta_{Thomas} = (\kappa, \mu, \sigma)$ corresponding to the parent intensity, offspring intensity, and cluster scale, respectively.

3.2 Persistent homology

Radially averaged statistics – such as the g and K functions – collapse a point cloud to a real-valued curve indexed by scale r . Features based on such statistics are constructed by evaluating over a chosen range $[r_{min}, r_{max}]$ and subsampling at n points along the interval to obtain a feature vector in $\mathbb{R}^n$. This approach is limited because it discards multi-scale connectivity information and void structure, i.e. *how* the density is distributed across the space. Persistent homology retains this information by tracking a family of simplicial complexes on the point cloud as the scale parameter grows, instead of a single scalar summary. Under a filtration such as Vietoris-Rips, the H_0 persistence summary records the order and scales at which clusters merge. The H_1 counterpart marks voids and gaps in the pattern as they persist across scales, allowing one to capture the clustering-induced emptiness or repulsion that a process may exhibit, which canonical statistics cannot see as clearly due to radial averaging. For each $h \in \mathbb{N}$, the h -dimensional persistence summary requires no closed form expression and no assumption about which process family generated it.

Throughout this section, let X be a point process on $\mathbb{R}^d$ and $W \subseteq \mathbb{R}^d$ a bounded window with $|X \cap W| = n$. Enumerate the points of $X \cap W$ to construct a point cloud $\mathbf{x} = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$. Let $\Sigma(\mathbf{x})$ denote the *full simplex* on $\mathbf{x}$, i.e. the collection of all non-empty subsets of $\mathbf{x}$.

Definition 3.4: Simplex A *simplex* of $\mathbf{x}$ is a non-empty finite subset $\sigma \subseteq \mathbf{x}$. Its faces are the non-empty subsets $\tau \subseteq \sigma$.

Definition 3.5: Simplicial complex A simplicial complex Σ on $\mathbf{x}$ is a collection of simplices closed under taking faces: $\sigma \in \Sigma, \tau \subseteq \sigma \implies \tau \in \Sigma$.

Definition 3.6 A filtration function on $\mathbf{x}$ is a monotone map $f : \Sigma(\mathbf{x}) \rightarrow \mathbb{R}^+$ such that

$$f(\tau) \leq f(\sigma) \quad \forall \quad \tau \subseteq \sigma.$$

Example Vietoris-Rips filtration function:

$$f_{VR}(\{x_i\}) = 0, \quad f_{VR}(\{x_i, x_j\}) = \|x_i - x_j\|,$$

extended to higher simplices by $f_{VR}(\sigma) = \max_{\{x_i, x_j\} \subseteq \sigma} f_{VR}(\{x_i, x_j\})$, i.e. the diameter of σ .

Example For $u \in \mathbb{R}^d$, let $N_k(u)$ denote its k nearest neighbours in $\mathbf{x}$. The DTM_k filtration function is

$$f_{\text{DTM}}(\sigma; k) = \max_{u \in \sigma} \left\{ \left(\frac{1}{k} \sum_{v \in N_k(u)} d(u, v)^2 \right)^{\frac{1}{2}} \right\}$$

again extended to higher simplices by the max-over-edges rule of the VR example. (?).

Claim 3.7. f_{DTM_k} is monotone, and is hence a valid filtration function (? , Prop. 3.5).

Definition 3.8 Given a filtration function f on $\mathbf{x}$ and $r \geq 0$, we construct

$$\mathcal{F}_r = \{\sigma \in \Sigma(\mathbf{x}) : f(\sigma) \leq r\}.$$

Monotonicity of f ensures each $\mathcal{F}_r$ is a simplicial complex and that $\mathcal{F}_s \subseteq \mathcal{F}_t$ for $0 \leq s \leq t$. The family $(\mathcal{F}_r)_{r \geq 0}$ is the *filtration* induced by f . We write $VR_r(\mathbf{x})$ for $\mathcal{F}_r$ under f_{VR} .

Definition 3.9: Betti number Fix $h \in \mathbb{N}$ and a simplicial complex Σ . Taking homology with coefficients in a field (we use $\mathbb{Z}_2$), $H_h(\Sigma)$ is a vector space of dimension $\beta_h = \dim H_h(\Sigma)$, the h -th *Betti number*. $\beta_0, \beta_1, \beta_2, \dots$ count connected components, loops, and cavities, respectively.

Definition 3.10: Persistence pair & lifetime For a filtration $(\mathcal{F}_r)_{r \geq 0}$, a h -dimensional homology class is *born* at the smallest scale $b \geq 0$ at which it appears in $H_h(\Sigma_b)$, and *dies* at the scale $d > b$ at which it merges into an older class or becomes a boundary. The pair (b, d) is a *persistence pair*, and $d - b$ its *lifetime*.

Example Consider H_0 . Every 0-dimensional class is born at $r = 0$ and dies when its two components merge, i.e. at the filtration value of the connecting edge.

Definition 3.11: Persistence diagram A persistence diagram $D_h = \{(b_i, d_i)\}_i$ is the multiset of dimension- h persistence pairs, plotted with birth on the x -axis and death on the y -axis.

Remark Every point of D_h lies above the diagonal $b = d$ because it must die after it is born. Distance from the diagonal is lifetime, so short-lived points near the diagonal are often treated as noise.

Definition 3.12: Tent function For a persistence pair (b_i, d_i) , the tent function is

$$\Lambda_i(t) = \max(0, \min(t - b_i, d_i - t)), \quad t \in \mathbb{R},$$

a triangular bump supported on $[b_i, d_i]$, peaking at height $(d_i - b_i)/2$ when $t = (b_i + d_i)/2$. Persistence landscapes and silhouettes are order statistics and weighted averages of this family; persistence images replace the tent with a smooth Gaussian bump over the same birth-persistence coordinates.

Definition 3.13: Persistence image The persistence image of D_h is constructed by:

1. Mapping to birth-persistence coordinates: $(b, d) \mapsto (b, d - b)$.
2. Weighting each point by $w(d - b)$.
3. Forming the persistence surface $\rho(z) = \sum_i w(d_i - b_i) \phi(z; (b_i, d_i - b_i))$, a weighted sum of isotropic Gaussian kernels with bandwidth σ_{im} centered at the transformed points.
4. Integrating ρ over a 64×64 pixel grid, giving a single-channel image $PI \in \mathbb{R}^{64 \times 64}$.

4 Problem Setup

Let X be a Neyman-Scott process on $\mathbb{R}^d$ with unknown parameter vector $\theta \in \mathbb{R}^{n_{\text{params}}}$. For example, the homogeneous Thomas process has $\theta = (\kappa, \mu, \sigma)$. We observe a single realization of X on a bounded window $W \subset \mathbb{R}^d$, giving a point cloud $\mathbf{x} = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ with $n = |X \cap W|$. The objective is to produce an estimate $\hat{\theta} = \hat{\theta}(\mathbf{x})$ of θ from $\mathbf{x}$ alone. We stress *single realization*: θ is not recovered from repeated draws of X , nor is $\mathbf{x}$ pooled or averaged across realizations. This is the setting a practitioner faces: one observed pattern and one chance to estimate the generating parameters. It is also what makes the problem hard in a way no estimator can fix: a fixed θ already induces a distribution over point clouds, so even the true parameter value gives rise to a spread of plausible $\mathbf{x}$, and it is this irreducible spread — not any weakness of a particular estimator — that sets the error floor characterized below.

The model is trained on a set of synthetic clouds $\{\mathbf{x}_i, \theta_i\}_{i=1}^{N_{\text{train}}}$ by backpropagating the RMS loss

$$\mathcal{L}(\hat{\theta}_i, \theta_i) = \sqrt{\sum_{j=1}^{n_{\text{params}}} (\hat{\theta}_i^j - \theta_i^j)^2},$$

where $\theta_i = (\theta_i^1, \dots, \theta_i^{n_{\text{params}}})$. The scale of each parameter varies [TODO: For example...]; therefore – if not normalized – the large-scale parameters will dominate this loss. To address this, we train in log-normalized parameter space,

$$\{\mathbf{x}_i, \theta_i\}_{i=1}^{N_{\text{train}}} \rightarrow \left\{ \mathbf{x}_i, \frac{\log(\theta_i) - \mu_{\log \theta}}{\sigma_{\log \theta}} \right\}_{i=1}^{N_{\text{train}}},$$

where – abusing notation – $\mu_{\log \theta}, \sigma_{\log \theta}$ denote the mean and standard deviation of the logged parameters over the training set.

We draw one realization per parameter vector. The dispersion of $\hat{\theta}$ across realizations sharing a common θ gives the irreducible error floor against which the model’s performance should be read. We name this quantity $\mathcal{L}_{\text{floor}}$: fix θ , draw R independent realizations $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(R)}$, fit $\hat{\theta}^{(r)}$ to each with a fast classical estimator (mincontrast by default), and compute $\mathcal{L}$ (Section 6.1) between $\{\hat{\theta}^{(r)}\}_{r=1}^R$ and the shared true θ ; averaging over a grid of θ spanning the design distribution gives $\mathcal{L}_{\text{floor}}$ itself. No estimator that sees only a single realization can average below $\mathcal{L}_{\text{floor}}$ over the same design distribution, since $\mathcal{L}_{\text{floor}}$ is realized by an estimator that already knows which θ generated the cloud and is degraded only by which realization of it was drawn. This is the lower reference point for every result in Section 7; the upper one, $\mathcal{L} \approx 1$, is defined in Section 6.1. Section 7.1 reports where the empirical estimate of $\mathcal{L}_{\text{floor}}$ currently stands.

5 Simulator and Experimental Design

Training data consists of pairs $(\mathbf{x}_i, \theta_i)$ in which θ_i is drawn from a design distribution over parameters space and each $\mathbf{x}_i$ is a single realization of the corresponding process. We describe the design distribution, sampling algorithm, and verification procedure used to generate the synthetic data.

5.1 Design distribution

The naive approach of sampling $(\kappa, \mu, \dots)$ uniformly and independently from intervals is inadequate. Point processes can typically be separated into qualitative regimes of interest, which occupy curved subsets of the parameter grid. Therefore uniform sampling would allocate most of its mass to configurations that are either near-Poisson or degenerate. We therefore sample in reparameterized coordinates that carry the same meaning for every family considered: [TODO: Condense, make general]

$\lambda = \kappa \mu$	expected offspring intensity,
κ	parent intensity,
$\nu = r\sqrt{\kappa}$	dimensionless overlap index,

where r denotes the cluster scale of the family in question: $r = \sigma$ for Thomas and $r = \sqrt{\sigma_1^2 + \sigma_2^2}$ for nested Thomas. The overlap index ν measures cluster radius against mean inter-parent spacing, so $\nu \ll 1$ gives well-separated clusters and $\nu \gtrsim 1$ gives clusters that merge (the realization approaches complete spatial randomness). Nested Thomas carries an additional coordinate, the scale ratio $\rho = \sigma_2/\sigma_1 \in (0, 1)$, which separates configurations in which both levels of structure are resolvable from those in which the process collapses to a single scale. [TODO: Make general. more coords?]

We draw $(\log \kappa, \log \lambda, \log \nu)$ uniformly over a grid and invert to recover $\mu = \lambda/\kappa$ and $r = \nu/\sqrt{\kappa}$ for the training labels. Log-uniform sampling is used because the parameters span several orders

of magnitude, and it matches the log-normalization applied to the ground-truth targets (see Section 4) so that training targets are approximately uniform in each coordinate. Draws are taken from a scrambled Sobol sequence. [TODO: what is sobol?]

The following constraints are applied to the design grid:

1. **Point budget.** The cost of computing persistence diagrams grows rapidly with the number of points, so we impose $\mathbb{E}[N] = \lambda \in [150, 800]$, fixing the range of $\log \lambda$.
2. **Clusters per window.** We require $\kappa|W| = \kappa \geq 15$ so that a single realization is informative about κ , and $\kappa|W| = \kappa \leq 120$ so that individual clusters remain resolvable.
3. **Overlap.** We impose $\nu \in [0.10, 0.90]$. The upper limit deliberately admits the near-Poisson regime so that the model is still exposed to configurations in which the parameters are weakly identifiable.

Parameter draws violating these constraint are rejected and redrawn before any cloud is generated. This filter acts on the design and not on realizations, therefore it does not distort the conditional law of $\mathbf{x}$ given θ . The realized acceptance rate is 24.8%.

5.2 Sampling algorithm

The instances of the Neyman-Scott construction may be generated with the following general framework. The components that change from instance-to-instance is the displacement kernel k and the offspring count distribution. For a window $W \subset \mathbb{R}^d$:

1. Expand W by an edge buffer b to produce W_{exp} , so that clusters whose parents lie outside W are not truncated.
2. Sample the parent process on W_{exp} : $n \sim \text{Poisson}(\kappa|W_{exp}|)$, with parents placed uniformly on W_{exp} .
3. For each parent p independently, draw an offspring count $n_p \sim \text{Poisson}(\mu)$.
4. For each offspring independently, draw a displacement $\epsilon \sim k$ and place the point at $p + \epsilon$.
5. Discard the parents and retain the offspring lying in W .

For the Thomas process $k = \mathcal{N}(0, \Sigma^2 I_d)$; for the Matérn cluster process k is uniform on the ball of radius R ; for nested Thomas the parent process at step 2 is itself a Thomas process rather than a homogeneous Poisson process.

The buffer is set by a single rule applied to whichever kernel is in use: b is the radius containing all but ϵ of the mass of k . For a Gaussian kernel this gives $b = 4\sigma$, at which the per-axis tail mass is $2\Phi(-4) \approx 6.3 \times 10^{-5}$; for the compactly supported Matérn cluster kernel it gives $b = R$ exactly, with no tail margin required; for nested Thomas the buffers of the two levels add, giving $b = 4\sigma_1 + 4\sigma_2$, which over-covers the tight bound $4\sqrt{\sigma_1^2 + \sigma_2^2}$ and is therefore conservative. Points are never clipped to the boundary; the only boundary operation is the containment filter at step 5. [TODO: Explain buffer better]

5.3 Validation

Throughout this section, let X denote a point process on $\mathbb{R}^d$. We write $u, v \in \mathbb{R}^d$ for points (precise locations) and du, dv for infinitesimal regions of volume $|du|, |dv|$ centered at u and v , respectively.

Definition 5.1: Second-order product density The second-order product density $\rho^{(2)} : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^+$ is defined such that $\rho^{(2)}(u, v) du dv$ is the probability of jointly observing a point of X in each of the infinitesimal regions du and dv .

Definition 5.2: Pair correlation function The pair correlation function $g(u, v) : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^+$ is defined

$$g(u, v) = \frac{\rho^{(2)}(u, v)}{\rho(u)\rho(v)}.$$

It is clear that for any Poisson process we have $g \equiv 1$. Intuitively, g can suggest whether SPPs are attractive ($g > 1$), or repulsive ($g < 1$).

Definition 5.3: Stationarity & isotropy A point process X is said to be:

- *Stationary* if $X \sim X + s$ for every $s \in \mathbb{R}^d$ where $X + s = \{x + s : x \in X\}$.
- *Isotropic* if $X \sim RX$ for every rotation R about the origin.

Note Stationarity implies constant intensity $\rho(u) = \lambda$, but the converse is not necessarily the case. If X is stationary and isotropic, $g(u, v)$ depends on u and v only through their inter-point distance $r = \|u - v\|$. Abusing notation, we may write $g(r)$ for this common value. We now define the K function.

Definition 5.4: K-function For stationary and isotropic X , the K -function $K(r) : \mathbb{R}^+ \rightarrow \mathbb{R}^+$ is defined

$$K(r) = \int_{b_r(0)} g(\|h\|) dh$$

Intuitively, $\rho(u)K(r)$ is the expected number of points we expect to see within a distance r of a typical point in X .

The validity of the proposed pipeline is dependent on a diligent cloud-generation process, which we validate against the established closed-form second-order theory (where possible). On $W = [0, 1]^2$ we compare 3000 realizations at each of three parameter triples spanning the design, chosen to span the overlap index: a tight-cluster regime ($\nu = 0.07$), a moderate regime ($\nu = 0.22$), and a near-Poisson regime ($\nu = 1.00$). [TODO: Explain better]

For the first moment, $\mathbb{E}[N(W)] = \kappa\mu|W|$. For the second, the canonical identity $\text{Var}[N(W)] = \kappa\mu(1 + \mu)|W|$ holds only in the limit of a window large relative to σ , and is badly biased in precisely the near-Poisson regime we wish to stress. Applying the law of total variance to $N(W) = \sum_p N_p(W)$ with $N_p(W) \mid p \sim \text{Poisson}(\mu q(p))$, $q(p) = \int_W k(x - p) dx$, and evaluating the shot-noise term by Campbell's formula gives the exact finite-window result [TODO: Explain better]

$$\text{Var}[N(W)] = \kappa\mu + \frac{\kappa\mu^2}{4\pi\sigma^2} J(\sigma)^2, \quad J(\sigma) = \int_{-1}^1 (1 - |t|) e^{-t^2/4\sigma^2} dt, \quad (1)$$

for $W = [0, 1]^2$, where the reduction to a one-dimensional integral follows from the triangular density of the difference of two independent $\text{Uniform}(0, 1)$ variables. As $\sigma \rightarrow 0$, $J(\sigma) \rightarrow 2\sigma\sqrt{\pi}$ and (1) collapses to $\kappa\mu(1 + \mu)$, confirming the two agree in the appropriate limit; it is (1), and not the canonical identity, that we use as the comparison target.

Case	ν	$\mathbb{E}[N(W)]$		$\text{Var}[N(W)]$	
		theory	empirical	theory	empirical
A	0.07	200.0	200.4 ± 0.6	982.0	972.5 ± 25.1
B	0.22	100.0	100.2 ± 0.4	545.2	544.8 ± 14.1
C	1.00	20.0	20.0 ± 0.1	43.6	46.0 ± 1.2

Table 1: Simulator validation against (1). All deviations are within 2 standard errors.

We additionally verify that the empirical pair correlation function and Ripley’s K match

$$g(r) = 1 + \frac{e^{-r^2/4\sigma^2}}{4\pi\kappa\sigma^2}, \quad K(r) = \pi r^2 + \frac{1 - e^{-r^2/4\sigma^2}}{\kappa},$$

that the marginal offspring intensity on W shows no boundary deficit (edge-to-interior density ratio 1.003–1.048 across the three cases, in all cases consistent with unity and never in the direction an insufficient buffer would produce), and that the displacement kernel is isotropic with per-axis standard deviation matching σ . Estimates of g normalized per realization carry a negative bias that grows with $\mathbb{E}[N^2]$ and is therefore larger for clustered patterns than for a matched-intensity Poisson control; we account for this when reading residual deviations rather than attributing them to the sampler.

6 Method: ParamNet

Let $\mathbf{x}$ denote a point cloud, v its vectorized persistence summary, and $e \in \mathbb{R}^{E+1}$ the resulting feature vector. ParamNet follows the pipeline **[TODO: define E]**

$$\mathbf{x} \xrightarrow{\text{Vectorization}} v \xrightarrow{\text{Encoder}} e \xrightarrow{\text{FCNN}} \hat{\theta}. \quad (2)$$

We note that the pipeline shape follows the neural-estimator patterns used in REF [Vihrs 2022] and REF. Our contribution lies in the feature-extraction stage, the application to a wider range of SPP families, and the careful evaluation of design choices.

The model is trained and evaluated on synthetic clouds generated according to the methodology described in Section 5. The synthetic clouds are processed together before training so that the feature-extraction step is performed only once.

Feature extraction is the model’s key component: it is where the cloud’s topological summary is vectorized and made usable for machine learning. A substantial body of work has developed vectorizations of persistence diagrams – persistence images, Betti curves, landscapes, kernel methods – that trade off stability, resolution, and information loss in different ways, but there has been no formalized discussion on which is preferable in this setting.

ParamNet uses the persistence image vectorization; the three alternatives we implemented, and their relative efficacies are reported in Section 7.3.5.

The vectorized summary v is passed through an encoder to obtain the feature vector $e \in \mathbb{R}^{E+1}$, which is then given to $NN_\phi : \mathbb{R}^{E+1} \rightarrow \mathbb{R}^{n_{\text{params}}}$ to produce the estimate $\hat{\theta}$. NN_ϕ is a small feed-forward network: two hidden layers of width 64 and 32, each followed by a ReLU nonlinearity and dropout ($p = 0.1$), then a final linear layer mapping to the n_{params} outputs.

Every experiment in this paper uses the same optimizer and schedule: AdamW (lr = 10^{-3} , weight decay 3×10^{-4}), batch size 32, up to 500 epochs with early-stopping patience 50 on the validation loss.

6.1 Splits and evaluation protocol

Data are split by parameter vector so that no parameter value appears in both training and evaluation. Beyond the random test split we generate two further evaluation sets: a grid over the design grid, permitting error to be reported as a surface over $(\log \nu, \log \lambda)$ rather than as a single scalar; and a set drawn one increment outside each range, characterizing degradation under extrapolation. **[TODO: Don't understand extra sets]**

The classical baselines introduced in Section 1 are run on the same clouds, under the same train/test/grid/extrapolation splits rather than on separate data or numbers taken from the literature. Section 7.2 is therefore a comparison against a shared test set.

Throughout Section 7 we report error in the log-normalized parameter space of Section 4. For a parameter vector with n_{params} components and a test set of size N_{test} , we define the per-parameter loss

$$\mathcal{L}^j = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} (\hat{\theta}_i^j - \tilde{\theta}_i^j)^2, \quad \mathcal{L} = \frac{1}{n_{\text{params}}} \sum_{j=1}^p \mathcal{L}^j,$$

where $\tilde{\theta}$ denotes the log-normalized parameter vector and $\hat{\theta}$ the model's estimate of it. Scalar figures quote $\mathcal{L}$; where a single number obscures the behaviour of an individual parameter we report the breakdown $\mathcal{L}^1, \dots, \mathcal{L}^{n_{\text{params}}}$ alongside it. **[TODO: Clean up]**

Two remarks. Firstly, $\mathcal{L}$ is the training objective of Section 4 up to the averaging over parameters and the square root, so the quantity being optimized and the quantity being reported differ only in scale; we report $\mathcal{L}$ rather than the training loss because the per-parameter decomposition is not available from the latter. Secondly, because the standardization is the one used at training time, $\mathcal{L}^j$ is measured in units of the training-set variance of $\log \theta^j$. A value of $\mathcal{L}^j = 1$ therefore corresponds to an error of one standard deviation of $\log \theta^j$ over the design distribution, which is the error of a model that has learned nothing beyond the mean of the design distribution. This gives the upper reference point for every figure reported in Section 7; the lower reference point is the irreducible error floor of Section 4.

6.2 Filtration and vectorization

We describe the pipeline implemented for the persistence image vectorization computed under the *DTM* filtration, which is the primary configuration carried through the rest of the paper. For an introduction to persistence images see Definition 3.13.

1. Pick filtration function and construct filtrations $\{(\mathcal{F})_{r \geq 0}\}_i$.
2. Compute persistence diagrams $\{D^{(h)}\}_i$.
3. Calibrate diagrams to determine common birth/persistence axis ranges, shared across the dataset.
4. Vectorize each diagram into a fixed-size raster using those ranges.

The key design choices that follow naturally from this pipeline are (1) the filtration function, (2) the calibration procedure, and (3) the image resolution. In particular, the choice of filtration function plays a fundamental role in what information is available to be extracted in the first place.

Instead of the canonical Rips or Čech filtrations, we use the DTM_k filtration (see Example 3.2). Rips is built from inter-point distances alone, making it sensitive to the sparsest connection between two regions and does not register how densely those regions are occupied. In this case DTM_k

is more suitable because it works with a point’s average distance to its k nearest neighbours; consequently, DTM_k provides a more direct measure of local density. Since the point process parameters directly affect the local density of the processes, we claim that this choice is natural. [TODO: Calibration issues for Rips]

The parameter k fixes the scale at which local density is read. Summaries computed at several values of k can be given to the encoder (with some additional architectural considerations) to provide the model with fine and coarse scales simultaneously. We compute a single scale, $k = 5$, noting that error grows monotonically in k on both Thomas and Nested Thomas. Fusing $k \in \{5, 10, 15\}$ does not improve on DTM_5 on Thomas, though it does on nested Thomas. Section 7.3.1 reports the relevant losses.

We compute the summary for H_0 only. H_0 records the order and scales at which clusters merge – the local density the parameters govern – while H_1 contributes nothing measurable beyond it under this vectorization and lands at chance level when it is used alone (Section 7.3.2).

Three alternative vectorizations – landscapes, silhouettes, Betti/Euler curves – are defined and compared in Section 7.3.5.

6.3 Calibration

Consider the training clouds $\{\mathbf{x}_i, \theta_i\}_{i=1}^{N_{train}}$ and fix the homology dimension $h \in \{0, 1\}$ for some choice of filtration. We obtain N_{train} persistence diagrams $\{D_i^{(h)}\}_{i=1}^{N_{train}}$. Each diagram will have a different distribution of points, some of which may have extreme outliers with respect to the full set of diagrams.

To ensure that in the resulting persistence images each pixel bears the same meaning, we calibrate the diagrams to have the same axis ranges. It is not immediately clear that naive approach of calibrating to the maximum deaths and births (thus including all points) is adequate in this context. This is because the resulting relative positioning of the bulk of points: if a diagram in the training set has strong outliers, the remaining diagrams will be rescaled in a way that places the majority of their points near the origin. Figure 1) demonstrates how this phenomenon might affect the vectorizations.

Our solution is to calibrate according to *per-diagram coverage quantiles*. For each training diagram $D_i^{(h)}$, define the three summary statistics

$$b_i^{\min} = \min_{(b,d) \in D_i^{(h)}} b, \quad b_i^{\max} = \max_{(b,d) \in D_i^{(h)}} b, \quad p_i^{\max} = \max_{(b,d) \in D_i^{(h)}} (d - b), \quad (3)$$

i.e. the smallest birth, largest birth, and largest persistence value *within a single diagram*. Given a coverage level $q \in (0, 1]$ and a padding factor $\alpha \geq 1$, we set the calibrated birth and persistence ranges to be the q -quantiles of these per-diagram statistics, taken over the training set:

$$[Q_{1-q}(\{b_i^{\min}\}_{i=1}^{N_{train}}), \alpha \cdot Q_q(\{b_i^{\max}\}_{i=1}^{N_{train}})] \quad \text{and} \quad [0, \alpha \cdot Q_q(\{p_i^{\max}\}_{i=1}^{N_{train}})], \quad (4)$$

where $Q_q(\cdot)$ denotes the empirical q -quantile. These two ranges are shared by every diagram of homology dimension h , for every point cloud in the dataset (train, validation, test, and adversarial alike), so that a given pixel of the resulting persistence image always refers to the same region of birth-persistence space.

This scheme is also compatible with the stability of persistence images REF [Adams 2017], which are stated for a fixed bandwidth and a fixed image domain: since σ_{pixels} is specified in pixel units (Definition 3.13) rather than in absolute birth/persistence units, a pixel remains a comparable, well-defined location across every image built from the calibrated imager.

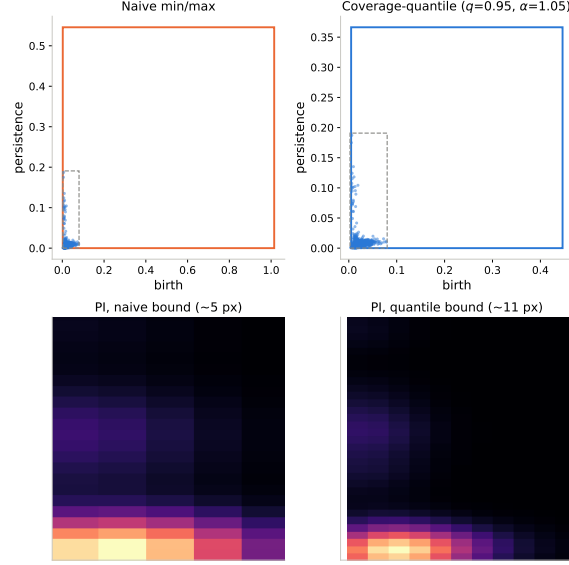


Figure 1: Naive pooled min/max calibration (left) versus per-diagram coverage-quantile calibration, $q = 0.95$, $\alpha = 1.05$ (right), for the same H_0 diagram (DTM filtration, $k = 5$, Thomas design distribution, Section 5). Top: the diagram against its calibrated birth-persistence box; the naive box is stretched by an outlier diagram elsewhere in the population, and the dashed square marks the region zoomed into below. Bottom: that region at each calibration’s native pixel density – a coarse $\sim 5 \times 5$ block of mostly-fused pixels under the naive bound versus a finer $\sim 11 \times 11$ grid resolving the secondary peak under the quantile bound, since the same 64×64 grid now covers a narrower range.

The calibrated ranges are fit *exclusively* on $\{D_i^{(h)}\}_{i \in \text{train}}$, then frozen and applied unchanged to validation, test, and adversarial diagrams. Fitting the ranges on the full, unsplit population of clouds leaks information about the held-out clouds into the training images.

For homology dimension H_0 , birth values are often identically zero (or otherwise carry negligible spread) under the chosen filtration, in which case $b_i^{\min} \approx b_i^{\max}$ for all i and the calibrated birth range collapses to a point. In these cases we fall back to a one-dimensional vectorization since the degenerate range would make an entire column of pixels convey no information.

The coverage-quantile rule is not specific to the persistence image: the same (q, α) scheme is applied to the corresponding per-diagram statistics of the 1-D grid underlying landscapes, silhouettes, and Betti/Euler curves (Appendix A).

The four calibration parameters – coverage q , padding α , kernel width σ_{pixels} , and raster resolution G – are swept one at a time around default values, each arm varying a single parameter while holding architecture and the remaining three fixed (Appendix A.1). Only $G = 128$ is unambiguously bad; q is the one parameter on which the two processes disagree (Thomas prefers $q = 0.99$ and Nested Thomas favors $q = 0.95$), and we keep the shared default $q = 0.95$ rather than tune it per-process. [TODO: Future work]

6.4 Encoder and head

Consider the training clouds $\{\mathbf{x}_i, \theta_i\}_{i=1}^{N_{\text{train}}}$. For each cloud we compute the H_0 persistence image of the DTM_5 diagram at resolution 64×64 . So for batch size B , the encoding stage is comprised of the following pipeline:

$$(B, 1, 64, 64) \xrightarrow{\text{Encoder}} (B, E) \xrightarrow{\text{Concatenation}} (B, E + 1)$$

In the encoder stage, the image is passed through a small convolutional stack: three 3×3 convolution blocks with channel widths (32, 64, 128), each followed by a ReLU and 2×2 max-pooling with spatial dropout ($p = 0.2$) between blocks (except the last one). The resulting feature map is reduced by adaptive average pooling to a single spatial location and projected by a linear layer to the embedding dimension E (we use $E = 64$) to produce a (B, E) tensor.

The concatenation step appends $\log N(\mathbf{x})$, the logarithm of the number of points in the cloud, to that embedding, giving a feature vector of dimension $E + 1$. The resulting $(B, E + 1)$ tensor is then passed to the FCNN head to produce the final parameter estimates.

With a single DTM_k scale only one encoder is necessary, and there is no need for an architecture that fuses multiple scales. Additional complexities arise when several scales are computed. We must choose whether to use a shared encoder (every scale is fed to an encoder with the same weights), or assign a separate encoder to each scale (thus increasing the number of encoder parameters threefold). Once the encoder framework is determined, a suitable method for fusing the per- k embeddings is necessary; for example simple concatenation, or more complex fusion architecture. The training results obtained from various such combinations are reported in Section 7.3.4.

7 Experiments

We first report the estimation performance of the model described in Section 6, together with the controls that make the numbers interpretable, and compare it against existing classical and neural baselines on shared test data. We then report the variations that were run and did not beat the current iteration of ParamNet.

A joint sweep over calibration parameters, architectures, and vectorizations is computationally unfeasible. Instead, we adopt a greedy sequence fixed in advance. Encoder and fusion mode are selected first (Section 7.3.4), against the persistence image at the default calibration ($q = 0.95$, Section 6.3). That architecture is then held fixed while the calibration parameters ($q, \alpha, \sigma_{\text{pixels}}, G$) are swept (Appendix A.1). A confirmation pass re-runs the two architectures that survive the fusion-mode sweep – shared and independent encoders under concatenation – at the selected calibration (Table 11), to check that the architecture choice was not an artifact of the default calibration under which it happened to be made. Vectorization-specific parameters (landscape K , silhouette p , etc.) are swept last, with calibration and architecture both frozen at their selected values (Section 7.3.5, Appendix A). **[TODO: Split up, repetition from calib section]**

Three conventions recur across every results table; we state them here to avoid unnecessary repetition in the captions. All reported losses are the squared error in log-normalized space defined in Section 6.1, averaged over parameters. A dagger (†) marks an arm on which at least one seed failed to converge and collapsed instead to the $\mathcal{L} \approx 0.9$ – 1.0 floor; in these cases the reported mean is dominated by the failed seed(s) and should not be read as the arm’s typical performances. **[TODO: Further work]**

7.1 Estimation performance

The model of Section 6 is assembled from choices each of which was measured one axis at a time against the multi-scale, $H_0 + H_1$, coordinate-channel configuration this work started from: dropping to DTM_5 , dropping H_1 , and dropping the coordinate channels each land within one seed-standard-deviation of the arm they were measured against on Thomas (Section 7.3). No run configures all three at once, so the figures quoted here and in Table 4 – $\mathcal{L} = 0.126 \pm 0.008$ on Thomas and $\mathcal{L} = 0.196 \pm 0.006$ on nested Thomas – belong to that starting configuration rather than to the

Arm	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
Full model (correct labels)	0.124 ± 0.007	0.196 ± 0.006
Shuffled labels	1.011 ± 0.024	0.997 ± 0.018

Table 2: Label-shuffle control, $n = 5$ seeds, both design distributions. Same architecture and calibration as the full model; only the training-label assignment is permuted.

Arm	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
Full model (PI, $H_0 + H_1$, $DTM\{5, 10, 15\}$)	0.124 ± 0.007	0.196 ± 0.006
log N removed	$0.305 \pm 0.392^\dagger$	0.200 ± 0.006
Topology removed (log N only)	0.891 ± 0.017	0.946 ± 0.018

Table 3: Bracketing the topological contribution: the full model against a model stripped of the log $N(\mathbf{x})$ feature (Section 6.4) and against a model with the topology removed entirely, keeping only log $N(\mathbf{x})$ fed straight to the FCNN head. Both design distributions, $n = 5$ seeds per arm. One of five Thomas seeds diverged to the design-distribution-mean floor ($\mathcal{L} \approx 1.0$, cf. Table 13); the other four land at 0.123–0.133, in line with the full model.

model as specified, and the end-to-end confirmation is outstanding. **[EXPT:]** Train and evaluate the headline configuration exactly as Section 6 specifies it – persistence image, H_0 only, DTM_5 , shared encoder, concatenation with log N , no coordinate channels, $q = 0.95$ – on both design distributions at $n = 10$ seeds, so that the reported $\mathcal{L}$ is the model the paper describes and not the configuration each simplification was measured against.

Two controls fix the scale on which those numbers should be read.

Sanity check: label shuffling. The † divergence in Table 13 is described as “no better than predicting the design-distribution mean,” but that floor was never independently measured. We do so directly: retraining the default PI model with training labels randomly permuted (inputs unchanged), so the only learnable target is each parameter’s marginal distribution. Both datasets land at $\mathcal{L} \approx 1.0$, confirming the † rows of Table 13 are genuinely collapsing to chance-level prediction and not to some other failure mode, and giving an empirical floor against which every other result in this section can be read.

Training on log N alone (i.e. no topological summary) collapses to the same $\mathcal{L} \approx 0.9$ –1.0 floor as the label-shuffle control of Table 2 on both datasets, so the point count carries almost none of the recoverable signal on its own. Conversely, removing log N from the full model costs little on nested-Thomas ($0.196 \rightarrow 0.200$) and is merely noisy on Thomas rather than clearly harmful (four of five seeds are indistinguishable from the full model; the fifth diverges). Together the two arms show the persistence images – not the point count – are carrying the signal, with log N a minor, possibly non-essential addition.

Performance relative to the irreducible error floor. Section 4 names $\mathcal{L}_{\text{floor}}$ – the dispersion of a fast classical estimator’s $\hat{\theta}$ across many realizations sharing one true θ – as the lower reference point every number in this section should ultimately be read against, complementing the upper reference point $\mathcal{L} \approx 1$ already carried as the bottom row of Table 4. $\mathcal{L}_{\text{floor}}$ itself does not yet have a value to report. `scripts/estimate_error_floor.py` was run on a dedicated $50\theta \times 200$ -realization Thomas dataset (`data/error_floor/thomas`, mincontrast estimator, 10 SLURM shards covering all 50 θ -groups): every shard completed without error (1000/1000 clouds fitted per shard, confirmed

in each shard’s own log) but every per-cloud loss reduced to NaN rather than a number, so all 50 groups – and hence the merged `floor_L` – are NaN. This looks like a bug in the script’s own aggregation step rather than a property of the estimator: the same `mincontrast` module, run on ordinary (non-repeated-realization) clouds above, completes without crashing on all 5 seeds of Table 4 even where 2 of those 5 land at elevated loss, so it is not silently producing NaN there. One concrete lead: the shard JSONs’ recorded `label_names` include `edge_buffer` and `c1`, names that do not belong to Thomas’s own (κ, μ, σ) target set, suggesting the per-target loss array is being indexed against the wrong label list for this dataset. Nested Thomas cannot take the same route at all under the current codebase: it has no closed-form K or g – the same closed-form dependency Section 1 raises against classical estimators generally – so `configs/runs/nested_thomas/error_floor.yaml` generates the repeated-realization dataset but nothing yet turns it into a floor estimate for that process. **[EXPT:]** Fix the label-indexing bug in `estimate_error_floor.py`’s aggregation and re-run for Thomas; nested Thomas needs either a derived closed form or a different per-realization floor estimator (e.g. `vihrs`, which needs no closed form) before the same route is even available. Until $\mathcal{L}_{\text{floor}}$ has a value, every loss in this section should be read only against the upper, chance-level reference point.

[EXPT:] Three evaluation sets defined in Section 6.1 are not yet reported and are each promised elsewhere in the paper: error as a surface over $(\log \nu, \log \lambda)$ on the grid set – expect degradation as $\nu \rightarrow 1$, where the parameters are weakly identifiable, and if the model degrades more gracefully than minimum contrast there, that IS the result; degradation on the out-of-range set one increment outside each design range; and the Matérn cluster runs for the cross-family comparison – whether one model trained across families beats per-family models is an interesting question in itself, and the common reparameterization of Section 5.1 is what makes it askable.

7.2 Comparison against baselines

We compare ParamNet (PI, $\text{DTM}_{\{5,10,15\}}$, Section 7.3.1) against the classical and neural baselines of Section 1, run on the identical clouds and train/test splits of Section 6.1 rather than against numbers taken from the literature: minimum contrast fit to Ripley’s K and, separately, to the pair correlation function g (Section 5.3), and `vihrs` – our re-implementation of the neural estimator conceded as prior work [A] in Section 2, Vihrs (2022): a 1-D CNN over the edge-corrected, centred Ripley $L(r) - r$ curve plus $\log N(\mathbf{x})$, i.e. the pipeline of Section 6 with the persistence-image feature replaced by a single classical radial summary and no persistence diagrams at all. Palm likelihood (?) is implemented (`cloudforger.baselines.palm`) but has not yet been run to completion; ### marks it below.

`vihrs` is not merely competitive with ParamNet on Thomas, it is better – and not only in aggregate. Its per-parameter losses ($\mathcal{L}^\kappa = 0.194 \pm 0.012$, $\mathcal{L}^\mu = 0.123 \pm 0.008$, $\mathcal{L}^\sigma = 0.029 \pm 0.002$) are each individually lower than ParamNet’s own (0.204 ± 0.010 , 0.132 ± 0.009 , 0.041 ± 0.006 ; Table 6’s $\text{DTM}_{\{5,10,15\}}$ row), so this is not a scalar-averaging artifact: for this single-level process, a hand-built radial summary plus a point count is not a straw-man baseline, and our topological pipeline does not beat it. The picture is different on nested Thomas, where ParamNet’s advantage in Table 4 is concentrated on the density/count targets – $\mathcal{L}^{\text{parent.intensity}} = 0.286$ vs. `vihrs`’s 0.314, $\mathcal{L}^{\text{meta.offspring}} = 0.420$ vs. 0.487, $\mathcal{L}^{\text{mean.offspring}} = 0.087$ vs. 0.111 – while `vihrs` keeps a clear edge on both scale parameters, $\mathcal{L}^{\text{meta.cluster.scale}} = 0.134$ vs. ParamNet’s 0.144 and $\mathcal{L}^{\text{cluster.scale}} = 0.029$ vs. 0.042. The scale/count split is consistent across both design distributions: $L(r) - r$ is built to resolve a characteristic radius directly, while a density-based topological feature is not, and it is only on the harder two-level process – where density and count are harder to read off a single radial curve – that this trade-off nets out in the topological pipeline’s favour overall.

Method	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
ParamNet (ours)	0.126 ± 0.008	0.196 ± 0.006
vihrs ($L(r) - r$ CNN)	0.115 ± 0.007	0.215 ± 0.004
Minimum contrast, K	0.827 ± 0.646^a	—
Minimum contrast, g	6.167 ± 0.331^b	—
Palm likelihood	###	###
<i>Chance-level reference (label shuffle, Table 2)</i>	1.011 ± 0.024	0.997 ± 0.018

Table 4: Estimator comparison, both design distributions, $n = 10$ seeds (ParamNet) or $n = 5$ seeds (vihrs, minimum contrast) on the shared test split of Section 6.1; ### = not yet run. Minimum contrast has no nested-Thomas entry because the process has no closed-form K or g to fit against – the same closed-form dependency Section 1 raises against classical estimators in general, here making the comparison itself impossible rather than merely inconvenient. The bottom row is the upper (chance-level) reference point of Section 6.1, repeated from Table 2, so every method above can be read against both 0 and chance. ^a Minimum contrast on K is unstable across seeds rather than uniformly bad: 3/5 seeds land at 0.379–0.399, close to vihrs, while 2/5 land far worse (0.903 and 2.056, the latter driven by a parent-intensity error of 3.24) – consistent with Section 1’s claim that minimum contrast is sensitive to the optimizer’s starting point specifically because the design distribution retains the near-Poisson regime where κ and μ are weakly identifiable. ^b Minimum contrast on g is, in contrast, consistently worse than chance on 5/5 seeds (5.82–6.66, tight relative to that mean) – a systematic bias rather than scattered non-convergence. Its own module (`cloudforgery.baselines.mincontrast_g`) documents that its contrast exponent $c = 1$ is set by analogy with ?’s K -based rule of thumb rather than tuned, and that the estimator has not been checked against ground-truth parameters the way the K -based fit implicitly has been (Section 5.3); we report the number rather than suppress it, but do not read it as a settled statement about contrast estimation on g in general.

Both neural methods dominate minimum contrast in either variant, by roughly an order of magnitude on Thomas and categorically on nested Thomas, where minimum contrast cannot even be attempted. Minimum contrast on g in particular lands *above* the chance-level floor of Table 4’s bottom row – worse than predicting the design-distribution mean – which Section 1’s critique of classical methods did not anticipate needed stating so starkly; see the table’s own caption for the caveats around that number before over-reading it.

Per-parameter breakdown. κ, μ, σ separately, for every method run on Thomas, are already laid out in Table 4’s discussion above and Table 6’s $\text{DTM}_{\{5,10,15\}}$ row; the same breakdown for the DTM_k filtration sweep itself (rather than across methods) is in Table 6 directly. The short version: $\mathcal{L}^\kappa$ (parent intensity) is the hardest target for every method on both processes by a wide margin, $\mathcal{L}^\sigma$ / $\mathcal{L}^{\text{cluster_scale}}$ the easiest, and this ordering holds regardless of which estimator is used – so the difficulty ranking across parameters looks like a property of the estimation problem itself (how much each parameter’s variation actually perturbs the observed point pattern) rather than an artifact of any one feature set or architecture.

7.3 Branches

Each of the following was implemented and run, and none of it is part of the model of Section 6. We report them in order of how close the discarded option sits to that model.

Method	$\mathcal{L}^\kappa$ (parent int.)	$\mathcal{L}^\mu$ (offspring int.)	$\mathcal{L}^\sigma$ (cluster scale)
ParamNet (ours)	0.204 ± 0.010	0.132 ± 0.009	0.041 ± 0.006
vihrs	0.194 ± 0.012	0.123 ± 0.008	0.029 ± 0.002
Minimum contrast, K	1.317 ± 1.009	0.912 ± 0.726	0.250 ± 0.202
Minimum contrast, g	8.792 ± 0.483	6.286 ± 0.355	3.425 ± 0.161

Table 5: Per-parameter breakdown of Table 4’s Thomas column ($n = 10$ ParamNet, $n = 5$ others). vihrs has the lowest loss in every column; see the discussion above for why this does not carry over to nested Thomas.

Filtration	$\mathcal{L}$ (mean $\pm$ sd)	$\mathcal{L}^\kappa$	$\mathcal{L}^\mu$	$\mathcal{L}^\sigma$	Failed seeds
Rips	— ^a	—	—	—	10/10 ^a
DTM ₅	0.125 ± 0.006	0.201 ± 0.009	0.134 ± 0.007	0.040 ± 0.004	0/10
DTM ₁₀	0.139 ± 0.008	0.221 ± 0.016	0.147 ± 0.008	0.049 ± 0.003	0/10
DTM ₁₅	0.146 ± 0.006	0.227 ± 0.011	0.155 ± 0.008	0.056 ± 0.010	0/10
DTM _{5,10,15} (default)	0.126 ± 0.008	0.204 ± 0.010	0.132 ± 0.009	0.041 ± 0.006	0/10

Table 6: Filtration comparison, Thomas process, persistence-image vectorization at the default calibration ($q = 0.95$, Section 6.3), $H_0 + H_1$, shared-weight CoordConv architecture (Section 6.4), $n = 10$ seeds per row (Rips: 10 attempted). $\mathcal{L}^\kappa, \mathcal{L}^\mu, \mathcal{L}^\sigma$ are the per-parameter breakdown (parent intensity, offspring intensity, cluster scale respectively; Section 6.1). ^a Rips fails identically on 10/10 seeds *before* training starts, not from a convergence failure: under Rips every H_0 feature is born at $r = 0$, so the calibrated birth range collapses to a point and the calibration step raises rather than silently degrading – exactly the degenerate-axis failure mode anticipated in Section 6.3. No loss is produced for this row by construction, and no DTM row shows a failed seed.

7.3.1 Multi-scale DTM fusion

We also compute the persistence image at several DTM scales at once and fuse the resulting embeddings. On Thomas this does not improve on the single scale $k = 5$, so the headline model does not carry it; on nested Thomas it does improve, and that exception is reported here rather than absorbed.

The n_k scales are handled by folding the scale axis into the batch dimension for a single forward pass – one call over $B \cdot n_k$ images rather than n_k separate calls – so the weights are tied across scales by construction rather than by an explicit sharing mechanism. The resulting (B, n_k, E) tensor is fused by concatenating the n_k embeddings along the feature axis before $\log N(\mathbf{x})$ is appended, giving $(B, n_k \cdot E + 1)$.

Filtration	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
Rips	— ^a	— ^a
DTM ₅	0.125 ± 0.006	0.205 ± 0.003
DTM ₁₀	0.139 ± 0.008	0.237 ± 0.007
DTM ₁₅	0.146 ± 0.006	0.260 ± 0.016
DTM _{5,10,15} (default)	0.126 ± 0.008	0.196 ± 0.006

Table 7: Table 6’s comparison repeated for both design distributions, scalar $\mathcal{L}$ only – nested Thomas’s five targets do not share Thomas’s κ, μ, σ decomposition. $n = 10$ seeds per row, except nested-Thomas’s DTM_{5,10,15} row, which reuses the $n = 5$ confirm_shared_concat run already reported as the PI/native entry of Table 12 and the shared-encoder row of Table 11, rather than a separately-seeded run. ^a Rips fails identically on both design distributions, 10/10 seeds each, for the reason given in Table 6’s caption.

Vectorization	Homology dims	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
PI	H_0 only	0.126 ± 0.006	0.194 ± 0.006
PI	H_1 only	0.895 ± 0.019	0.911 ± 0.013
PI	H_0+H_1 (default)	0.124 ± 0.007	0.196 ± 0.006
LS	H_0 only	0.135 ± 0.006	0.246 ± 0.002
LS	H_1 only	###	###
BC	H_0 only	0.237 ± 0.012	0.294 ± 0.014
BC	H_1 only	0.260 ± 0.019	0.510 ± 0.009

Table 8: Homology-dimension ablation (C3), $n = 5$ seeds per arm, both design distributions, native encoder path at the shared $q = 0.95$ calibration (Section 6.3) rather than each vectorization’s own appendix-preferred default. PI and BC are complete; LS’s H_1 -only arm has not been run (### = not yet run) – LS’s H_0+H_1 default is the native row of Table 12 (0.132 ± 0.008 T / 0.250 ± 0.005 NT, 3 seeds), not repeated here since it was not re-run at $n = 5$.

Error grows monotonically with k within a single scale, on both design distributions: DTM_5 is the strongest individual scale and DTM_{15} the weakest (Thomas: $0.125 \rightarrow 0.146$; nested Thomas: $0.205 \rightarrow 0.260$, a proportionally larger degradation), consistent with a finer neighbourhood retaining more of the local-density signal the parameters govern while a coarser one averages exactly that structure away. Fusing all three scales tells two different stories across the two processes. On Thomas, the fused arm (0.126 ± 0.008) does not improve on the single best scale, DTM_5 (0.125 ± 0.006): the two are within one seed-standard-deviation of each other, so fusion’s advantage there is not raw accuracy but not having to select k in advance. On nested Thomas the fused arm (0.196 ± 0.006) clearly beats every single scale tried, including DTM_5 (0.205 ± 0.003) – the two-level design benefits from combining scales in a way the single-level one does not, plausibly because the meta- and sub-cluster structure sit at genuinely different characteristic radii that no single k resolves together. Dropping to the single scale is therefore free on Thomas and costs 0.009 on nested Thomas, and it is the one place where the model of Section 6 is a trade rather than a simplification.

Rips fails outright under this vectorization on both processes, for the degenerate-birth reason given in Section 6.2; every other result in Section 7.3.5 and Appendix A is accordingly reported only under DTM .

7.3.2 Homology dimensions

We also compute the H_1 diagram alongside H_0 . For the persistence image it adds nothing measurable to H_0 , and on its own it collapses to the label-shuffle floor, so the headline model uses H_0 only.

For the persistence image, H_1 alone lands within noise of the label-shuffle floor (Table 2) on both datasets, while H_0 alone is statistically indistinguishable from the fused H_0+H_1 default – essentially all the recoverable signal is in H_0 (connected-component birth/death, i.e. local density) and H_1 (loop structure) contributes nothing measurable beyond it on either design distribution. The Betti curve does not replicate this collapse: BC’s H_1 -only arm (0.260 T / 0.510 NT) is worse than its H_0 -only arm (0.237 T / 0.294 NT) but nowhere near the ≈ 0.9 – 1.0 label-shuffle floor that PI’s H_1 -only arm collapses to, on either dataset. A feature-count curve therefore retains some usable signal in the loop structure that a persistence-image raster, at this calibration, does not – though BC’s H_0 -only number is itself well above PI’s, so this is not a claim that BC’s H_1 channel is more informative in absolute terms.

Arm	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
Full model (CoordConv on)	0.124 ± 0.007	0.196 ± 0.006
CoordConv removed	0.126 ± 0.005	0.194 ± 0.002

Table 9: CoordConv ablation: the shared-weight encoder of Section 6.4 with and without the two coordinate channels, at the default $q = 0.95$ calibration, $n = 5$ seeds per arm on both design distributions.

Encoder mode	Fusion	$\mathcal{L}$ (mean $\pm$ sd)	Failed seeds
shared	concat	0.122 ± 0.007	0/5
shared	conv-avg	$0.431 \pm 0.425^\dagger$	2/5
shared	conv-flatten	$0.589 \pm 0.425^\dagger$	3/5
independent	concat	0.129 ± 0.006	0/5
independent	conv-avg	0.133 ± 0.015	0/5
independent	conv-flatten	0.141 ± 0.016	0/5

Table 10: Full encoder-mode $\times$ fusion-mode cross, $n = 5$ seeds per combination, on Thomas at the default PI calibration. “Failed” seeds are those that collapse to the $\mathcal{L} \approx 0.9$ design-distribution-mean floor of Table 2 rather than converging. † mean is dominated by the failed seeds and is not a stable estimate of the arm’s typical performance; see per-seed values in the text.

7.3.3 CoordConv

We also augment each persistence image with two coordinate channels, on the reasoning that calibration is what makes a pixel position mean something and the network should be handed that rather than have to recover it. The ablation does not support the reasoning, so the headline model does not use them.

During the processing step each $(D, 64, 64)$ persistence-image stack is augmented with two coordinate channels – fixed x - and y -coordinate grids linearly spaced over $[-1, 1]$ – following the CoordConv construction REF [Liu et al., CoordConv 2018], so that the convolutional stack has access to absolute pixel position rather than having to infer it. This ensures that the work done during the calibration step is not lost: each pixel has a fixed, shared meaning across the dataset and CoordConv lets the network exploit this directly instead of relearning it.

CoordConv removal is within seed noise of the full model on both datasets – if anything marginally better on Thomas and on nested-Thomas. This does not support the claim that CoordConv is necessary to exploit the calibrated pixel grid; at the current calibration ($q = 0.95$, Section 6.3) the convolutional stack appears to recover comparable information without explicit coordinate channels.

7.3.4 Encoder and fusion mode

We also cross encoder mode against fusion mode. Flat concatenation under a shared encoder is both the simplest arm and the only one that trains reliably, which is what Section 6.4 adopts – though with a single scale the encoder-sharing axis of this cross does not arise at all.

The architecture sweep was run on the persistence-image arm at the default $q = 0.95$ calibration; Section 7.3.5 confirms PI is the arm worth ablating.

The sweep crosses *encoder mode* (shared-weight vs. independent-per- k) against *fusion mode* (concatenation, average-pool, or a small convolutional-flatten fusion head), five seeds per combination, on Thomas.

Encoder mode	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
shared, concat	0.124 ± 0.007	0.196 ± 0.006
independent, concat	0.133 ± 0.005	0.228 ± 0.029

Table 11: Cross-dataset confirmation of the shared-vs-independent encoder choice, $n = 5$ seeds per arm.

[FIGURE:] Per-seed loss scatter across the six configurations – the collapse pattern below is the interesting part and this table’s means alone understate it.

Restricted to concatenation, shared and independent encoders match to within seed noise (0.122 ± 0.007 vs. 0.129 ± 0.006); since sharing weights across scales reduces the parameter count (roughly K -fold) and removes a design axis, we adopt the shared encoder going forward.

Both convolutional-fusion variants are unstable, but *only when paired with the shared encoder*: shared/conv-avg and shared/conv-flatten each collapse to the $\mathcal{L} \approx 0.9$ floor on 2/5 and 3/5 seeds respectively (individual per-seed losses: $\{0.128, 0.120, 0.902, 0.115, 0.892\}$ and $\{0.903, 0.118, 0.903, 0.130, 0.892\}$), while the identical fusion heads under an independent-per- k encoder converge cleanly on all five seeds every time (0.133 ± 0.015 , 0.141 ± 0.016) without matching plain concatenation.

Flat concatenation is therefore preferred not only for its simplicity but because – together with the shared-encoder counterpart – it is the only combination that trains reliably and reaches the best loss. The same two concatenation arms (shared and independent encoder) were re-run on nested-Thomas to check the choice transfers across design distributions:

On nested-Thomas the shared encoder’s advantage over the independent one is larger than on Thomas and the independent arm is noisier (± 0.029 vs. ± 0.005), so the shared-encoder choice seems to matter even more on the more difficult five-parameter design distribution, reinforcing the choice made above.

7.3.5 Alternative vectorizations

We also implemented three alternative vectorizations of the same diagrams – landscapes, silhouettes, and Betti/Euler curves. None of them improves on the persistence image, on either process or either encoder path, so the persistence image is the summary carried through the paper. Figure 2 gives a visual overview of the vectorizations, computed from the same persistence diagram; we describe each in turn and report their respective performance in Table 12.

Landscape. The K layers $\lambda_1(t) \geq \lambda_2(t) \geq \dots \geq \lambda_K(t)$ are the order statistics of the tent family $\{\Lambda_i(t)\}_i$ (Definition 3.12) at each grid point t : $\lambda_j(t)$ is the j -th largest tent value. K is chosen per diagram sample by a coverage rule – the smallest K for which layer $K + 1$ is negligible for 99% of training diagrams, capped at 16 – giving a (K, G) raster per homology dimension.

Silhouette. Rather than keep every order statistic, the silhouette collapses the same tents into one persistence-weighted average,

$$\phi^{(p)}(t) = \frac{\sum_i |d_i - b_i|^p \Lambda_i(t)}{\sum_i |d_i - b_i|^p},$$

a single $(1, G)$ curve, with the amplitude exponent p fixed at 1 here.

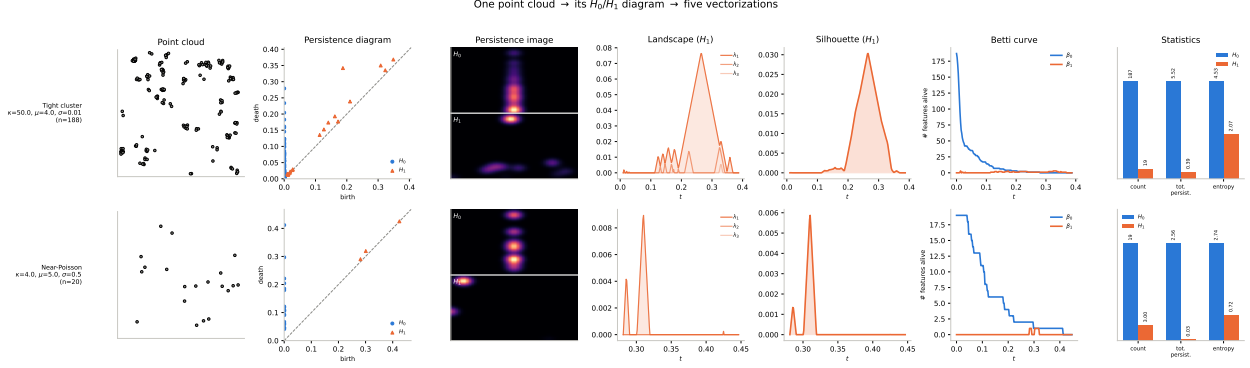


Figure 2: One point cloud $\rightarrow$ its H_0/H_1 persistence diagrams $\rightarrow$ the vectorizations compared in Section 7.3.5: persistence-image raster, landscape stack, silhouette curve, Betti curve, and persistence-statistics summary. Top row: the tight-cluster design ($\kappa = 50$, $\mu = 4$, $\sigma = 0.01$); bottom row: the near-Poisson design ($\kappa = 4$, $\mu = 5$, $\sigma = 0.5$), the same two extremes of Table 1. Statistics-panel bars are min-max normalized per statistic, with raw values printed above each bar, since count, total persistence, and entropy live on incomparable scales.

Vec.	dim/channel	channels	$\mathcal{L}$, native encoder		$\mathcal{L}$, MLP encoder	
			T	NT	T	NT
PI	$64 \times 64 = 4096$	6	0.126 ± 0.008 (10)	0.196 ± 0.006 (5)	0.154 ± 0.012 (5)	0.261 ± 0.011 (5)
LS	$16 \times 128 = 2048$	6	0.132 ± 0.008 (3)	0.250 ± 0.005 (3)	0.167 ± 0.019 (5)	0.284 ± 0.009 (5)
SIL	128	12	0.165 ± 0.013 (3)	0.271 ± 0.012 (3)	0.177 ± 0.013 (5)	0.297 ± 0.004 (5)
BC	128	6	0.196 ± 0.014 (3)	0.290 ± 0.005 (3)	0.176 ± 0.009 (5)	0.266 ± 0.003 (5)

Table 12: Vectorization methods comparison. “Native” denotes the methods’ purpose-built encoder (CNN for PI/LS, 1-D CoordConv for SIL/BC); “MLP” is the architecture-agnostic FlattenMLPEncoder applied to the same tensor. All native-path arms use the $q = 0.95$ calibration matched to the persistence image rather than each vectorization’s own tuned default from Appendix A, so the comparison is calibration-fair; BC’s own appendix-preferred $q = 0.99$ scores slightly better (0.179 ± 0.012 T / 0.284 ± 0.011 NT) but is not used here for consistency. SIL is fixed at $p = 1$ throughout (the SIL-1 arm of Appendix A.3) for both the native and MLP path. T = Thomas, NT = nested-Thomas; seed count n in parentheses.

Betti/Euler curve. The Betti curve $\beta_k(r) = \#\{i : b_i \leq r < d_i\}$ counts k -features alive at scale r – a feature count rather than a tent reduction, but built from the same persistence pairs. The Euler curve $\chi(r) = \beta_0(r) - \beta_1(r)$ is a signed count nested inside the two Betti channels, not additional information.

For a discussion of each vectorization’s calibration (grid ranges, K -selection, shared vs. per-dimension axes) refer to Appendix A; for per-stage tensor shapes refer to Appendix B.

The persistence image is the strongest arm on both design distributions and on both encoder paths, and the margin survives the substitution of the architecture-agnostic MLP encoder for each vectorization’s own, so the ranking is not an artifact of the encoder each summary happens to be paired with.

8 Discussion and Limitations

Notes, not prose. Each bullet is one paragraph to write.

- **Negative results.** Three alternative vectorizations were implemented and none improved

on persistence images (Section 7.3.5). Say plainly what this does and does not license – it is evidence about these summaries under this architecture and this design distribution, not a general claim about vectorizations. The same qualification covers the other branches of Section 7.3: multi-scale fusion, H_1 , and coordinate channels each fail to earn their place *here*.

- **Where the simplification costs something.** Fused scales beat DTM_5 on nested Thomas (0.196 vs. 0.205, Section 7.3.1), so the single-scale model is a trade on the two-level process and not a free saving; say so rather than let the Thomas result stand for both.
- **Calibration inherited across summaries.** The per-diagram coverage rule was inherited unchanged by the 1-D summaries for comparability, which may not be optimal for each individually.
- **Compute cost.** [TABLE:] Feature-extraction time per cloud by vectorization, training time, and amortized inference time per cloud against minimum contrast per cloud. The last comparison strongly favours the method and is currently unmentioned.
- **The design distribution is a prior.** The design distribution acts as a prior, so the trained model approximates a posterior summary under it, and its ranges must therefore cover the intended application domain. This is the single most important limitation of any amortized estimator and deserves a full paragraph, not a trailing sentence.
- **Computational cost.** Persistence computation scales badly in n ; this is why the point budget of Section 5.1 is capped at 800. State the cost explicitly and compare against the cost of minimum contrast.
- **Single realization.** The estimator sees one realization; classical methods often assume the same, but say so explicitly.
- **Homogeneity and stationarity.** Everything here assumes homogeneous, stationary, isotropic processes.

9 Future Work

Notes, not prose. Sequenced as written.

- **Immediate.** Complete Section 7.1: the headline confirmation run, the error floor, the grid surface, the extrapolation set, and the Matérn cluster runs. Give this a timeline – it is the nearest-term deliverable.
- **Near term.** The encoder question: attention over scales, and set-based encoders operating on diagrams directly rather than on rasters.
- **Medium term.** Inhomogeneous / non-stationary processes, where classical closed forms are least available and the topological approach has the most to gain – this is arguably where the thesis-scale contribution lies.
- **Medium term.** Real data. Galaxy catalogues if the running example of Section 1 is cosmological; name a specific dataset.
- **Longer term.** Uncertainty quantification – the model currently emits a point estimate with no interval, which limits its use as a drop-in replacement for likelihood-based methods.

Swept parameter	Value	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
Coverage q	0.90	0.125 ± 0.008	0.195 ± 0.004
	0.95 (default)	0.126 ± 0.006	$0.441 \pm 0.341^\dagger$
	0.99	0.122 ± 0.006	0.203 ± 0.009
	1.00 (naive)	0.120 ± 0.005	0.210 ± 0.004
Padding α	1.00	0.123 ± 0.007	0.201 ± 0.008
	1.05 (default)	0.126 ± 0.007	0.199 ± 0.007
	1.10	0.125 ± 0.006	0.202 ± 0.009
Kernel width σ_{pixels}	0.5	0.124 ± 0.007	0.192 ± 0.008
	1.0	0.123 ± 0.006	0.202 ± 0.013
	2.0	0.124 ± 0.009	$0.457 \pm 0.329^\dagger$
Resolution G	32	0.122 ± 0.004	0.209 ± 0.011
	64 (default)	0.125 ± 0.009	0.199 ± 0.007
	128	$0.635 \pm 0.352^\dagger$	$0.684 \pm 0.339^\dagger$

Table 13: Calibration sweep for the persistence-image (PI) vectorization, one-at-a-time around the default calibration ($q = 0.95$, $\alpha = 1.05$, $\sigma_{\text{pixels}} = 0.75$, $G = 64$; Section 6.3), for both the Thomas and nested-Thomas design distributions (Section 5). Diagrams use the default $DTM_{\{5,10,15\}}$, $H_0 + H_1$ pipeline and shared-weight architecture of Section 6.4. Cells report $\mathcal{L}$ (Section 6.1, mean $\pm$ sd over 3 seeds). † marks configurations that diverge on at least one of the three seeds (per-seed $\mathcal{L} \approx 0.9\text{--}1.0$, i.e. no better than predicting the design-distribution mean); their mean is dominated by the diverged seed(s) and should not be read as a stable estimate.

A Calibration Sweeps

Section 6.3 discusses the calibration for the persistence image’s birth/persistence axes by per-diagram coverage quantiles fit on the training split alone. The same (q, α) scheme – coverage q and padding α over per-diagram $b^{\min}, b^{\max}, p^{\max}$ – carries over unchanged to the 1-D grid $[t_{\min}, T]$ underlying landscapes, silhouettes, and Betti/Euler curves, since all three are built from the same tent family $\{\Lambda_i(t)\}_i$ (Definition 3.12) rather than the 2-D birth-persistence surface. This appendix reports, per vectorization – the persistence image included – the calibration sweeps run at the multi-scale architecture of Section 7.3.1: a CoordConv-augmented CNN encoder with shared weights across DTM scales $k \in \{5, 10, 15\}$ followed by concatenation fusion, and a two-layer MLP head. Every sweep below therefore predates the simplifications of Section 6 and was run under the branch configuration, not under the headline model.

A.1 Persistence Images

We quantify the effect of the calibration parameters introduced in Section 6.3: coverage q , padding α , kernel width σ_{pixels} , and raster resolution G . Table 13 reports the sweep, each arm varying a single parameter around the default calibration while holding architecture and the remaining three parameters fixed.

[FIGURE:] $\mathcal{L}$ against q , both design distributions, with $q = 1.00$ marked as the naive endpoint. One-parameter family, so a line plot reads better than a table and makes the dataset disagreement below visible at a glance.

Excluding the † rows, resolution $G = 128$ is the only arm that is unambiguously bad – $G \in \{32, 64\}$, α , and σ_{pixels} all vary within one seed-standard-deviation of the default across both

Swept parameter	Value	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
Grid resolution G	64	0.135 ± 0.015	0.249 ± 0.003
	128 (default)	0.137 ± 0.008	0.240 ± 0.003
	256	0.141 ± 0.014	0.246 ± 0.002
K -rule threshold τ	0.5%	0.134 ± 0.008	0.245 ± 0.007
	1% (default)	0.135 ± 0.010	0.248 ± 0.006
	2%	0.146 ± 0.026	0.243 ± 0.006
Fixed K (vs. data-derived)	4	0.174 ± 0.013	0.372 ± 0.023
	8 (budget-matched, $G = 512$) [†]	0.154 ± 0.011	0.312 ± 0.007
	16	0.132 ± 0.012	0.244 ± 0.006
	data-derived (default)	0.133 ± 0.007	0.245 ± 0.006

Table 14: Calibration sweep for the persistence-landscape (LS) vectorization, one-at-a-time around the native-budget default ($G = 128$, $\tau = 0.01$, K data-derived), on the *params* task for both the Thomas and nested-Thomas design distributions (Section 5). Diagrams use the default $DTM_{\{5,10,15\}}$, H_0+H_1 pipeline and shared-weight architecture of Section 6.4 – the same one used for the persistence-image sweep of Table 13 – with two landscape channels per scale (six total). The diagram-grid coverage $q = 0.95$ (Section 6.3) is fixed across every arm; K -selection uses coverage $c = 0.99$ and cap $K_{\max} = 16$, both fixed except in the third group. Cells report $\mathcal{L}$ (Section 6.1, mean $\pm$ sd over 3 seeds). [†] the $G = 512$, $K = 8$ arm matches the 4096-d persistence-image budget of Table 13 exactly; the other three rows of that group hold $G = 128$ fixed and vary only K , so this row is not raster-size-matched to them.

datasets. Coverage q is the one parameter with a genuine process-dependent disagreement: the Thomas optimum moves toward the naive endpoint $q = 1.00$ (0.120 vs. 0.126 at the default), while nested-Thomas prefers tighter coverage at $q = 0.90$ (0.195) and gets markedly worse as $q \rightarrow 1$. We choose to keep the shared default $q = 0.95$.

The [†] rows are the one behaviour the calibration does not explain. $G = 128$ fails on at least one of three seeds on *both* design distributions, which looks like an optimization failure at the increased raster size rather than a calibration effect, and the same reading applies to the single diverged $\sigma_{\text{pixels}} = 2.0$ seed on nested Thomas; neither is a statement about the calibrated range.

A.2 Persistence Landscapes

For homology dimension h and DTM scale k , the landscape layers $\lambda_1(t) \geq \lambda_2(t) \geq \dots \geq \lambda_K(t)$ are the order statistics of $\{\Lambda_i(t)\}_i$ at each grid point t (Section 7.3.5), giving a (K, G) raster per (h, k) that is fed to the encoder as a non-square image (height K , width G) – the only structural difference from the $G \times G$ persistence image. K is not hand-picked: a coverage rule selects, per (h, k) , the smallest K such that

$$\|\lambda_{K+1}\|_{\infty} \leq \tau \cdot \|\lambda_1\|_{\infty}$$

for at least a fraction c of training diagrams, capped at $K_{\max}$; the per-dimension answers are then reconciled to their shared maximum so that the H_0 and H_1 channels can be stacked into one raster. Three calibration axes are therefore specific to this vectorization and are not present in the persistence-image sweep: the grid resolution G , the rule’s own relative threshold τ (fixed at 0.01 throughout every other experiment in this paper), and K itself, which we also sweep as a fixed override against letting the rule choose it. Table 14 reports all three, one-at-a-time, holding $c = 0.99$, $K_{\max} = 16$, and the diagram-grid coverage $q = 0.95$ fixed except where swept.

The coverage rule resolved to $K = 16$ – the cap – on every arm and seed underlying the first

Swept parameter	Value	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
Grid resolution G	64	0.169 ± 0.008	0.268 ± 0.006
	128 (default)	0.165 ± 0.011	0.271 ± 0.009
	256	0.170 ± 0.013	0.273 ± 0.012

Table 15: Grid-resolution calibration sweep for the silhouette (SIL-1, $p = 1$) vectorization, on the *params* task for both the Thomas and nested-Thomas design distributions (Section 5). Diagrams use the default $DTM_{\{5,10,15\}}$, H_0+H_1 pipeline and shared-weight architecture of Section 6.4 – the same one used for the persistence-image sweep of Table 13. The diagram-grid coverage $q = 0.95$ and padding $\alpha = 1.05$ (Section 6.3) are fixed across every arm at the values selected for the persistence image. Cells report $\mathcal{L}$ (Section 6.1, mean $\pm$ sd over 3 seeds).

two groups, so the G and τ sweeps report robustness at a K already pinned by $K_{\max}$, not by τ itself; their near-flat $\mathcal{L}$ (differences mostly within one seed-sd) is consistent with sweeping a quantity (G, τ) that is not presently the binding constraint on K . The fixed- K group is the informative one for this rule: the “data-derived” row lands within seed noise of the explicit $K = 16$ row on both datasets, exactly as expected since both resolve to the identical ($G = 128, K = 16$) configuration, while $K = 4$ and $K = 8$ are both markedly and monotonically worse. Within the range tested, more landscape layers help and $K_{\max}$ – not τ – is the binding constraint; distinguishing the rule’s own threshold from its cap would require re-running with a larger $K_{\max}$.

A.3 Silhouettes

The silhouette collapses the same tent family $\{\Lambda_i(t)\}_i$ into the single persistence-weighted curve $\phi^{(p)}(t)$ of Section 7.3.5, sampled on the same coverage-quantile grid $[t_{\min}, T]$ (Section 6.3) at resolution G ; both the normalized curve and its unnormalized numerator are kept as separate channels (the denominator discards the feature count, which the numerator preserves), giving $2 \times 2 \times 3 = 12$ 1-D channels (H_0+H_1 , $DTM\ k \in \{5, 10, 15\}$) fed to a per-scale 1-D CoordConv-style encoder under the same shared-weight, concat-fusion architecture as Table 13. Unlike K for landscapes, the amplitude exponent p has no data-derived rule to calibrate: Section 7.3.5 treats it as an experimental arm compared against $\{0, 2, 3\}$ elsewhere, not a calibration choice, so this appendix fixes $p = 1$ (the SIL-1 arm) and reports only the grid-resolution sweep, holding $q = 0.95$ and $\alpha = 1.05$ (Section 6.3) fixed at the values selected for the persistence image.

Resolution has little effect in either direction: every arm is within roughly one seed-sd of every other on both design distributions, and no configuration shows the divergence behavior marked [†] in Table 13. $G = 64$ is a reasonable default going forward – it costs a quarter of $G = 256$ ’s raster and is not measurably worse on either dataset.

A.4 Betti and Euler Curves

The Betti curve $\beta_k(r)$ and Euler curve $\chi(r) = \beta_0(r) - \beta_1(r)$ of Section 7.3.5 are sampled on the same coverage-quantile grid as the other 1-D vectorizations, giving a (grid.size,) curve per homology dimension and DTM scale. The runs below use χ disabled (two Betti channels stacked, H_0+H_1 , no derived Euler channel), so the channel count matches SIL-1 and the landscape sweep – 2 dims $\times$ 3 scales = 6 channels – rather than the $H_0+H_1+\chi$ default used elsewhere in the paper. A Betti curve has no layer-count axis, so there is no K -analogue to calibrate here, and its values are already bounded (small alive-feature counts, not raw pixel magnitudes), so grid resolution G and

Swept parameter	Value	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
Grid resolution G	64	0.183 ± 0.009	0.284 ± 0.006
	128 (default)	0.200 ± 0.004	0.289 ± 0.002
	256	0.196 ± 0.012	0.293 ± 0.008
Coverage q	0.95 (default)	0.190 ± 0.008	0.291 ± 0.003
	0.99	0.178 ± 0.009	0.285 ± 0.007
	1.00 (naive)	0.187 ± 0.009	0.344 ± 0.002

Table 16: Calibration sweep for the Betti-curve (BC) vectorization, one-at-a-time around the native-budget default (grid_size = 128, $q = 0.95$), on the *params* task for both the Thomas and nested-Thomas design distributions (Section 5). Diagrams use the default $DTM_{\{5,10,15\}}$, $H_0 + H_1$ pipeline (Euler channel disabled, see text) and shared-weight architecture of Section 6.4. Padding is fixed at $\alpha = 1.05$ throughout – the value selected for the persistence image – rather than this vectorization’s own code default of $\alpha = 1.1$. Cells report $\mathcal{L}$ (Section 6.1, mean $\pm$ sd over 3 seeds).

calibration coverage q are the two axes swept, one at a time, at the shared multi-scale architecture of Table 13.

$G = 64$ is again at least as good as $G = 128$ on both design distributions – the same pattern as SIL-1 above, consistent with a curve-shaped vectorization having little to gain from a finer grid once the underlying feature count is small. Coverage behaves differently than it does for the persistence image: $q = 0.99$ is the best arm on both datasets, and $q = 1.00$ (the naive, uncalibrated range) is clearly worse on nested-Thomas – a $\sim 21\%$ increase in $\mathcal{L}$ over $q = 0.99$, with a seed-sd tight enough (± 0.002) that this is a real effect and not a diverged seed. Unlike the † rows of Table 13, no arm here shows per-seed divergence to the design-distribution mean; $q = 1.00$ is simply a worse calibrated range, not a failure mode.

B Tensor Shapes

The persistence-image path of Section 6.4 runs $(M_i, 2)$ birth–death pairs $\rightarrow$ one 64×64 raster per diagram $\rightarrow (B, 1, 64, 64) \rightarrow (B, E)$ out of the encoder $\rightarrow (B, E + 1)$ at the head input; the multi-scale variant of Section 7.3.1 carries the extra n_k axis folded into the batch dimension, giving $(B \cdot n_k, D + 2, 64, 64) \rightarrow (B, n_k, E) \rightarrow (B, n_k \cdot E + 1)$ with the two coordinate channels of Section 7.3.3 included.

[TABLE:] One stage-by-stage shape table per vectorization, from $(M_i, 2)$ birth–death pairs through to the head input, in the notation of Section 6.4 (B, n_k, D, G, E) . The landscape (K, G) raster, silhouette two-channel curve, Betti curve and statistics vector paths need the same treatment. Flag in the landscape row that the raster is non-square, which is why the coordinate-channel builder needs independent height and width ramps.